Daily Quant Digest: 2026-10-28

Foundations Restart, day 3. Two bricks are down in each track. Today: how to count without listing, how a price is really two prices, and how to rearrange an array in place.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 3: counting I: the multiplication principle, arrangements (permutations), factorials	Easy
Derivatives	FR Lesson 3: how a market works: bid and ask, market makers, long and short, leverage	Easy
Algorithms & DS	FR Lesson 3: arrays, scanning and in-place updates; next permutation	Easy-Medium
Music	The Rolling Stones, "Gimme Shelter" (1969)	

1. Probability & Statistics: Counting Without Listing

> **Where we are.** Lesson 1: when all outcomes are equally likely, $P(A) = |A|/|\Omega|$, people in the group divided by people in the room. Lesson 2: combine groups with "or", "and", "not", and fix double counts with inclusion-exclusion. Both lessons needed us to **count** the room. With two dice (36 cells) we could list everything. With a shuffled deck we cannot. Today we learn to count rooms far too big to list.

1.1 The central picture: a road with forks

Picture a walk through a park where the path keeps forking. At the first fork you have 3 choices. Whichever you take, the next fork offers 4 choices. How many different complete walks are there?

You do not need to walk them all. Each of the 3 first choices opens its own set of 4 second choices: 4 walks, plus 4, plus 4. That is **4 for each of the 3**, so $3 \times 4 = 12$ walks. Draw it as a tree: 3 branches, each splitting into 4 twigs, 12 twig tips. Each tip is one complete walk.

In the picture	In the maths
A fork in the path	One step of a choice (a “stage”)
The number of paths leaving a fork	The number of options at that stage
A complete walk, start to finish	One outcome
The tips of the tree	The whole room Ω
“Count the tips without walking”	The **multiplication principle**

1.2 The multiplication principle

In words: if a choice is made in stages, and stage 1 has n_1 options, and **whatever you picked before**, stage 2 has n_2 options, stage 3 has n_3, and so on, then the total number of outcomes is

$$n_1 \times n_2 \times n_3 \times \cdots$$

Read aloud: “multiply the number of options at each stage.”

With numbers. An FX order ticket has three fields: the currency pair (5 choices), the side (buy or sell, 2 choices), and the date it settles (4 choices). The number of different tickets is $5 \times 2 \times 4 = 40$. Listing them would take a page; multiplying takes a second.

The key condition is in bold above: the **number** of options at each stage must not depend on what you picked earlier. The options themselves may change (we will see this in a moment); only their count must stay fixed.

A trading-floor scene. A bank wants to give each of its 1,000 traders a unique short code made of capital letters. Two letters? $26 \times 26 = 676$ codes: not enough, some traders would share. Three letters? $26 \times 26 \times 26 = 17{,}576$: plenty. Now add a rule, “no letter repeated in a code”. The first letter has 26 options, the second has 25 (anything except the first), the third has 24. The second letter’s options **change** with your first pick (if you picked A, B is allowed; if you picked B, it is not), but there are **always 25** of them. So the rule still applies: $26 \times 25 \times 24 = 15{,}600$ codes.

1.3 Arrangements: filling chairs one at a time

Now the most important special case. You have 5 different trade tickets and want to stack them in a pile. How many orders are possible?

Fill the pile one position at a time, like filling chairs in a row:

- Chair 1 (top of the pile): any of the 5 tickets.
- Chair 2: any of the 4 left.
- Chair 3: any of the 3 left.
- Chair 4: 2 left.
- Chair 5: 1 left, no choice.

So $5 \times 4 \times 3 \times 2 \times 1 = 120$ orders. Each ordering is called an **arrangement** or **permutation**.

This product of "n, times one less, times one less, … down to 1" appears so often that it has its own symbol, the **factorial**:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1.$$

Read aloud: "n factorial." **With numbers:** $3! = 6$, $5! = 120$, $10! = 3{,}628{,}800$.

Arranging only some of them. A bank ranks its 10 traders by P&L and publishes only the gold, silver and bronze. How many different podiums? Fill only 3 chairs: $10 \times 9 \times 8 = 720$. In factorial language this is $10!$ with the unused tail $7 \times 6 \times \cdots \times 1 = 7!$ divided away:

$$P(n,k) = n \times (n-1) \times \cdots \times (n-k+1) = \frac{n!}{(n-k)!}, \qquad P(10,3) = \frac{3{,}628{,}800}{5{,}040} = 720.$$

Symbols: n = how many items you have, k = how many chairs you fill, $P(n,k)$ = the number of ordered ways to do it.

Why is $0! = 1$? Put $k = n$ (fill every chair): the formula gives $n!/0!$, and we know the answer is $n!$. So $0!$ must be 1. It also makes sense in words: there is exactly one way to arrange nothing (do nothing).

1.4 How fast factorials grow

n	2^n	$n!$
5	32	120
10	1,024	3,628,800
20	1,048,576	about 2.4×10^{18}
52	about 4.5×10^{15}	about 8.07×10^{67}

A shuffled deck can be in about 8×10^{67} orders. That is so many that every properly shuffled deck in history has, almost surely, been in an order nobody has ever seen before. Keep this table in mind for the Algorithms section: any method that tries all $n!$ orders dies somewhere around $n = 12$.

1.5 Turning counts into probabilities

The multiplication principle counts the room **and** the group, then Lesson 1 divides. Example: 5 tickets stacked at random (all 120 orders equally likely). What is the chance the EURUSD ticket is on top? Group: fix EURUSD on top, arrange the other 4 below it: $4! = 24$. So $P = 24/120 = 1/5$. Of course: each of the 5 tickets is equally likely to be on top. When a symmetry argument gives the same answer, you know your counting is right.

Where the analogy stops working. The tree picture assumes each stage's choices can be counted without looking ahead. Some problems break this ("arrange so that no two neighbours are equal"), because the number of good options at a stage depends on earlier choices. Then you split into cases (Lesson 2's "or") or count the complement.

1.6 Common mistakes

- **Adding instead of multiplying.** "3 shirts and 4 ties gives 7 outfits." No: each shirt pairs with each tie, $3 \times 4 = 12$. Add when you choose **one stage or the other**; multiply when you make **one choice and then another**.
- **Using the principle when the count of options changes.** "How many 3-digit numbers have all digits distinct?" Starting from the last digit invites errors with the leading zero. Start with the most restricted stage: first digit 9 options (not 0), second 9 (anything but the first, zero now allowed), third 8: $9 \times 9 \times 8 = 648$.
- **Forgetting whether order matters.** A podium (gold, silver, bronze) cares about order: 720. A "top-3 team" where order does not matter has fewer: each team appears $3! = 6$ times among the 720 podiums, so $720/6 = 120$ teams. That division is tomorrow's lesson (combinations).

1.7 Self-check

1. How many ways can 7 different books be arranged on a shelf?
2. How many 4-digit numbers (1000 to 9999) have all four digits different?
3. 8 runners in a race. How many possible (gold, silver, bronze) results, and what is the probability that a given runner wins gold?

1.8 Interview question: the coffee queue

(A warm-up of the kind asked at SIG and Optiver. It tests the "glue" trick and symmetry.)

Six traders, A, B, C, D, E and F, line up in a random order at the coffee machine (all orders equally likely).

a. What is the probability that A and B stand next to each other?

b. What is the probability that A stands **directly** in front of B?

c. What is the probability that A is somewhere in front of B, and B is somewhere in front of C (not necessarily next to each other)?

1.9 Solution

The room. Six people in six chairs: $6! = 720$ equally likely queues.

(a) Glue A and B together. Treat "A and B as a pair" as one big block. Now there are 5 things to arrange: the block, C, D, E, F. That is $5! = 120$ orders. Inside the block, A and B can stand as AB or BA: 2 ways. Multiplication principle: $120 \times 2 = 240$ queues.

$$P(\text{adjacent}) = \frac{240}{720} = \frac{1}{3}.$$

(b) Same glue, but now the block must be AB, only 1 inner order: 120 queues, $P = 120/720 = 1/6$.

(c) Look only at the positions of A, B and C. Whatever spots they occupy, the 3 people can be placed in those spots in $3! = 6$ orders, all equally likely, and exactly 1 of them is "A, then B, then C". So $P = 1/6$. By counting: $720/6 = 120$ queues.

Interview trick. For (a), think about A's neighbour instead. Fix A's spot. If A is at one of the 2 ends (probability $2/6$), A has 1 neighbour, and B is that neighbour with probability $1/5$. If A is in the middle (probability $4/6$), A has 2 neighbours: probability $2/5$. Total: $\frac{2}{6} \cdot \frac{1}{5} + \frac{4}{6} \cdot \frac{2}{5} = \frac{2}{30} + \frac{8}{30} = \frac{10}{30} = \frac{1}{3}$. An even quicker way uses today's $P(n, k)$: A and B take some ordered pair of spots, $P(6, 2) = 6 \times 5 = 30$ equally likely choices. The neighbouring ones are (1,2), (2,3), (3,4), (4,5), (5,6) and the same five reversed: 10. So $P = 10/30 = 1/3$, and the 5 un-reversed ones give (b): $5/30 = 1/6$.

A nice surprise. (b) and (c) have the same answer, $1/6$, for completely different reasons. In an interview, saying why both are $1/6$ is worth more than the numbers.

Sanity check. I listed all 720 queues by computer: 240 have A and B adjacent, 120 have A directly in front of B, 120 have A before B before C. Also, (b) must be half of (a), since "adjacent" splits evenly into AB and BA. It is.

Self-check answers. (1) $7! = 5{,}040$. (2) First digit 9 options (not zero), then 9, 8, 7: $9 \times 9 \times 8 \times 7 = 4{,}536$. (3) $8 \times 7 \times 6 = 336$ podiums; a given runner wins gold in $7 \times 6 = 42$ of them, so $42/336 = 1/8$ (symmetry: any of 8 runners is equally likely to win).

---

2. Derivatives: How a Market Works

> **Where we are.** Lesson 1: money grows when lent ($P(1 + rt)$, Pe^{rt}). Lesson 2: future money is worth less today, $\text{PV} = \text{cash flow} \times DF(t)$. Both lessons assumed one price for everything. Today we meet the real market, where every price is really **two** prices, and we learn what it means to be long, short and leveraged.

2.1 The scene

A Hong Kong importer must pay a German supplier **EUR 10 million**. It holds dollars, so it calls its bank: "Where can I buy EUR 10 million against dollars?" The bank's FX dealer answers instantly: **"1.1650 / 1.1652."**

Who is on each side? The importer is a **price taker**: it needs euros for its business and accepts the price shown. The bank is a **market maker**: it stands ready all day to buy from anyone who wants to sell and sell to anyone who wants to buy. It does not care which way

the price goes. It wants to earn a small fee on each trade, and get rid of the risk quickly.

2.2 The central picture: the airport money-changer

You have seen this booth. Its board shows two prices for each currency: "US dollars: **we buy** at HKD 7.70, **we sell** at HKD 7.90". A market maker is exactly that booth, just with tighter prices and bigger tickets.

At the airport booth	In the market	Name
"We buy" (lower price)	The price the dealer pays you	**Bid**
"We sell" (higher price)	The price the dealer charges you	**Ask** (or **offer**)
The gap between them	The dealer's fee	**Spread** = ask minus bid
Halfway between	The "fair" reference price	**Mid** = (bid + ask) / 2
The booth's stock of notes	The dealer's open risk	**Inventory** (position)

In "1.1650 / 1.1652" the price is "dollars per 1 euro". The bid 1.1650 is where the bank **buys** euros from you; the ask 1.1652 is where it **sells** euros to you. The rule to remember: **you always buy at the higher price and sell at the lower one.** The quoter always wins the spread. (The last digit of an FX quote has a special name, the pip. That is tomorrow.)

Our importer needs to buy euros, so it "lifts the offer" (buys at the ask): it pays $10{,}000{,}000 \times 1.1652 = 11{,}652{,}000$ dollars.

2.3 Why does a spread exist at all?

Think of a second-hand car dealer. It buys your car for less than it will sell it for. Two reasons:

1. **Inventory risk.** The car sits in the yard for a while. Prices might fall before it sells. The dealer charges for holding that risk.
2. **The lemon problem.** Some sellers know their car is bad. The dealer cannot tell who, so it pays everyone a little less. In markets this is called **adverse selection**: some of the people trading with you know something you do not (a big order coming, news not yet priced in). The spread protects the dealer against them.

So the spread is wide when the market is jumpy (inventory risk is large) or when news is due (more people may know more). It is tight when the market is calm and full of competing dealers. EURUSD, the most traded currency pair in the world, has some of the tightest spreads anywhere; a rarely traded currency can cost many times more. Spread as a percentage in our quote: $0.0002/1.1651 \approx 0.017\%$.

2.4 Long and short

After the trade, look at both sides.

- The importer now **owns** euros. It is **long** EUR: it gains if the euro rises (its euros are worth more dollars) and loses if it falls.
- The bank **sold** euros it was not holding, so it now **owes** euros. It is **short** EUR 10 million: it gains if the euro falls (it can buy back cheaper) and loses if it rises.

Analogy for short. You borrow a friend's book, sell it for 100, and promise to return the same book later. If the price drops to 80, you buy a copy, return it, and keep 20. If it rises to 130, you must pay 130 to buy it back and you lose 30. Being short means you owe something, so you want it to get cheaper. Notice the danger: a long position can lose at most what you paid (the price cannot go below zero), but a short position has **no ceiling** on its loss, because the price has no ceiling.

2.5 The market maker's day: a worked trade

Mid is $(1.1650 + 1.1652)/2 = 1.1651$. Follow the bank through three scenes.

Scene 1: the spread earned. The bank sold EUR 10m at 1.1652, which is 0.0001 above mid. It has already "earned" $10{,}000{,}000 \times 0.0001 = 1{,}000$ USD **compared with the fair price**. But it is short EUR 10m, and that is not profit until the risk is gone.

Scene 2: the lucky ending. Ten minutes later a fund calls and **sells** EUR 10m to the bank at the bid, 1.1650. The bank's inventory is back to zero: it sold at 1.1652 and bought at 1.1650. Profit: $10{,}000{,}000 \times (1.1652 - 1.1650) = 2{,}000$ USD, with no risk left. This is the dream: capture the whole spread on a round trip.

Scene 3: the unlucky ending. Instead, news hits before any seller calls, and the euro jumps. The bank finally buys back EUR 10m at 1.1700:

$$\text{P\&L} = 10{,}000{,}000 \times (1.1652 - 1.1700) = -48{,}000 \text{ USD}.$$

Read it as: "amount, times (price I sold at minus price I bought at)". One bad move wiped out the spread from 24 round trips. That is why market makers rarely sit and wait. Two tools: they **hedge** quickly by buying from another dealer (paying the spread away), or they **skew** their quote. If the bank is short euros and wants buyers to stay away and sellers to come, it moves both prices up, say to 1.1651 / 1.1653. Now its bid is the most attractive in town for sellers, and its offer is less attractive for buyers.

2.6 Leverage

Leverage means controlling a position bigger than your own money, by borrowing the rest.

Analogy: a mortgage. You buy a 10 million flat with 1 million of your own money and 9 million borrowed. If the flat rises 2% to 10.2 million, you made 0.2 million on 1 million of your own: **+20%**. If it falls 10% to 9 million, you still owe 9 million, and your 1 million is **gone**. Leverage 10 times means every move in the asset is 10 times bigger for **your** money.

$$\text{return on your money} \approx \text{leverage} \times \text{asset return}, \qquad 10 \times 2\% = 20\%, \quad 10 \times (-10\%) = -$$

Read aloud: “what you make is the move, multiplied by how many times your money you are controlling.” The useful flip side: at leverage L, a move of $1/L$ against you wipes you out. At 20 times, a 5% move; at 50 times (common for retail FX accounts in some countries), a 2% move.

When it went wrong: 15 January 2015. The Swiss National Bank had held EURCHF at or above 1.20 for over three years, so many retail traders bet on it with high leverage, believing the floor made it safe. That morning the SNB removed the floor without warning. EURCHF fell from 1.20 to around 0.85 within minutes, roughly 29%, and for a while there were almost no bids at all: dealers pulled their quotes, so the spread became enormous. Traders with 50 times leverage lost many times their deposits. The broker FXCM was left with about USD 225 million of client losses it could not collect and needed an emergency rescue loan; Alpari UK went into insolvency. Lesson: leverage plus a market where the bid disappears is how accounts go below zero.

Where the analogy stops working. The airport booth sets its own prices and rarely changes them. A real market maker updates its quotes thousands of times a day, competes with dozens of other dealers, and manages one big inventory across all its clients, so it can offset one client’s buy against another’s sell.

2.7 Likely interview questions

1. **You quote EURUSD 1.1650 / 1.1652 and three clients in a row buy from you. What do you do?** You are now short and the flow says buyers are keen, so the price may be about to rise. Skew both prices up (maybe widen too), and hedge part of the position in the interdealer market.
2. **What does it cost a client to buy EUR 10m and sell it straight back at 1.1650 / 1.1652?** The full spread: $10{,}000{,}000 \times 0.0002 = 2{,}000$ USD.
3. **You run 25 times leverage. What move wipes you out?** About $1/25 = 4\%$ against you.

---

3. Algorithms & Data Structures: Next Permutation

> **Where we are.** Lesson 1: Big-O, count steps, and $n!$ is far worse than n^2. Lesson 2: `std::vector`, `std::string`, references, loops, `std::reverse` on a range, and the read/write trick for editing in place. Today: **scanning an array and updating it in place**. And a link to the Probability section: today we count permutations, and here we walk through them one at a time.

3.1 The central picture: a car's mileage counter

Watch a car's odometer tick from 1,999 to 2,000. What happened? Reading from the right, the digits 9, 9, 9 were already as large as they can be. So the first digit from the right that **can** grow, the 1, grows to 2, and everything to its right resets to the smallest value, 000.

Next permutation is the same machine, with one twist: you may only **rearrange** the digits you already have.

On the odometer	In next permutation
The tail of 9s, maxed out	The tail that is **decreasing** (reading left to right): it cannot grow
The first digit from the right that can grow	The **pivot**
The digit grows by the smallest step	Swap the pivot with the **smallest bigger** number in the tail
Everything to the right resets to 0s	The tail is put in its **smallest** order (increasing)

3.2 The problem

Given an array of integers, rearrange it into the **next larger arrangement in dictionary order**. If it is already the largest arrangement, rearrange it into the smallest (sorted increasing). Do it **in place**, using $O(1)$ extra memory.

"Dictionary order" means compare the first numbers; if they tie, compare the second; and so on, just as a dictionary orders words.

Constraints: $1 \le n \le 10^5$, values may repeat.

Examples:

- `[1, 2, 3]` gives `[1, 3, 2]`.
- `[3, 2, 1]` gives `[1, 2, 3]` (the largest wraps around to the smallest).
- `[1, 1, 5]` gives `[1, 5, 1]`.
- `[1, 3, 5, 4, 2]` gives `[1, 4, 2, 3, 5]`.

3.3 Brute force first

Make a list of **every** arrangement, sort the list, find ours, return the one after it. For 3 numbers that is fine: 6 arrangements, in order

`123, 132, 213, 231, 312, 321`.

The answer after `123` is `132`; the answer after `321` wraps to `123`. Correct, but there are $n!$ arrangements, each of length n: $O(n! \cdot n)$ time and memory. From the Probability table, $n = 20$ already gives about 2.4×10^{18} arrangements. Hopeless for $n = 10^5$.

3.4 The insight: change as little as possible, as far right as possible

The next arrangement must be **bigger**, but **only just**. A change on the left makes a big jump (like changing the thousands digit); a change on the right makes a small one. So change the **rightmost** position you can.

Work through `[1, 3, 5, 4, 2]`.

Step 1: find the tail that cannot grow. Scan from the right: 2, then 4 (bigger), then 5 (bigger). Reading left to right the tail `5, 4, 2` is decreasing. A decreasing tail is already the **largest** order of those numbers, like 999 on the odometer. No rearrangement of the tail alone makes it bigger. Keep scanning: 3 is **smaller** than its right neighbour 5. Stop. The pivot is the 3 at index 1.

Step 2: find what the pivot should become. The pivot must grow, but by the smallest step. Among the tail `5, 4, 2`, the numbers bigger than 3 are 5 and 4; the smallest of these is **4**. Because the tail is decreasing, the smallest bigger number is simply the **rightmost** number bigger than the pivot: scan from the right end and stop at the first number greater than 3.

Step 3: swap them. `[1, 3, 5, 4, 2]` becomes `[1, 4, 5, 3, 2]`.

Step 4: reset the tail to its smallest order. After the swap the tail `5, 3, 2` is **still decreasing** (we put 3 where the 4 was, and 3 also fits between 5 and 2). Its smallest order is increasing, so just **reverse** it: `2, 3, 5`. Result: `[1, 4, 2, 3, 5]`. Done.

Why step 4 can reverse instead of sort. Sorting costs $O(n \log n)$. But the tail is decreasing before and after the swap, so reversing it gives increasing order in $O(n)$. This is the beautiful part of the algorithm.

No pivot? If the whole array is decreasing (`[3, 2, 1]`), it is the largest arrangement. Step 4 alone reverses the whole array into `[1, 2, 3]`, the smallest. The wrap-around comes for free.

Duplicates. With `[1, 1, 5]` we must be careful with “bigger”: the scan in step 1 uses `>=`, so equal neighbours count as part of the tail, and step 2 uses “strictly bigger”. Then the pivot is the second 1, it swaps with 5, and we get `[1, 5, 1]`.

Complexity. Each of the three scans (find pivot, find swap partner, reverse) walks part of the array once: $O(n)$ time. Only a few integer variables: $O(1)$ extra space.

3.5 C++ solution

```cpp
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <vector>

// Brute force: list every arrangement in dictionary order, find ours, take the next.
```

```cpp
std::vector<int> nextPermutationBrute(const std::vector<int>& a) {
    std::vector<int> sorted = a;
    std::sort(sorted.begin(), sorted.end());
    std::vector<std::vector<int>> all;              // every distinct arrangement
    do {
        all.push_back(sorted);
    } while (std::next_permutation(sorted.begin(), sorted.end()));
    for (std::size_t k = 0; k < all.size(); ++k) {
        if (all[k] == a) return all[(k + 1) % all.size()];  // last wraps to first
    }
    return a;  // never reached
}

// The real solution: O(n) time, O(1) extra space, edits the caller's vector.
void nextPermutation(std::vector<int>& a) {
    const int n = static_cast<int>(a.size());
    // Step 1: scan from the right past the non-increasing tail to find the pivot.
    int i = n - 2;
    while (i >= 0 && a[i] >= a[i + 1]) --i;
    if (i >= 0) {
        // Step 2: rightmost number in the tail that is strictly bigger than the pivot.
        int j = n - 1;
        while (a[j] <= a[i]) --j;
        // Step 3: swap them.
        std::swap(a[i], a[j]);
    }
    // Step 4: the tail is still non-increasing; reverse it into increasing order.
    // With no pivot (i == -1) this reverses everything: wrap to the smallest.
    std::reverse(a.begin() + (i + 1), a.end());
}

int main() {
    struct Case { std::vector<int> input; std::vector<int> expected; };
    const std::vector<Case> cases = {
        {{1, 3, 5, 4, 2}, {1, 4, 2, 3, 5}},
        {{1, 2, 3}, {1, 3, 2}},
        {{3, 2, 1}, {1, 2, 3}},
        {{1, 1, 5}, {1, 5, 1}},
        {{2, 3, 3, 1}, {3, 1, 2, 3}},
        {{7}, {7}},
    };
    for (const auto& c : cases) {
        std::vector<int> a = c.input;   // copy, because nextPermutation edits it
        nextPermutation(a);
        assert(a == c.expected);
        assert(nextPermutationBrute(c.input) == c.expected);
    }
    // Stepping through all 4! = 24 arrangements of {1,2,3,4} brings us back home.
    std::vector<int> w = {1, 2, 3, 4};
    int steps = 0;
    do { nextPermutation(w); ++steps; } while (w != std::vector<int>{1, 2, 3, 4});
    assert(steps == 24);

    std::mt19937 rng(20261028);                     // random cross-check
    for (int t = 0; t < 50000; ++t) {
        int n = 1 + static_cast<int>(rng() % 7);
        std::vector<int> a(n);
        for (int& x : a) x = static_cast<int>(rng() % 4);  // many duplicates
        std::vector<int> mine = a, lib = a;
        nextPermutation(mine);
        std::next_permutation(lib.begin(), lib.end());
        assert(mine == lib);
        if (t < 5000) assert(mine == nextPermutationBrute(a));
    }
    std::cout << "All tests passed.\n";
```

```
    return 0;
}
```

The full file is `2026-10-28.cpp` (it also prints each example and runs a fully decreasing array of 100,000 numbers, the worst case). Compile with `g++ -std=c++17 -O2 -Wall 2026-10-28.cpp`. It passes all six examples, the 24-step cycle, and 50,000 random arrays with repeated values, agreeing with both the brute force and the standard library.

3.6 Walkthrough of the C++

`void nextPermutation(std::vector<int>& a)`. A reference **without** `const`, as in yesterday's version 3: the function edits the caller's own vector, and returns nothing. That is what "in place" looks like in C++. In `main` we copy first (`std::vector<int> a = c.input;`) so the test case itself is not changed.

`const int n = static_cast<int>(a.size());` `size()` returns an **unsigned** type (`std::size_t`). We convert to a signed `int` because our index `i` must be allowed to reach -1. With unsigned numbers, `0 - 1` wraps around to a huge positive value, and the loop `while (i >= 0 ...)` would never stop. This is the same trap as yesterday's count-down loop.

`while (i >= 0 && a[i] >= a[i + 1]) --i;` A loop with no body except `--i`. Short-circuit evaluation (from yesterday) matters: when `i` reaches -1, the test `i >= 0` is false, so C++ never reads `a[-1]`. If you swapped the two conditions, the program would read outside the array: undefined behaviour, the most dangerous kind of C++ bug, because it may "work" on your machine and crash in production.

`while (a[j] <= a[i]) --j;` No bounds check is needed here. We only get here when a pivot exists, which means `a[i + 1] > a[i]`, so the scan is guaranteed to stop at or before `j = i + 1`. Knowing **why** a loop must stop is half of writing safe C++.

`std::swap(a[i], a[j]);` Exchanges two values. It takes its arguments by reference, so it changes the actual array slots.

`std::reverse(a.begin() + (i + 1), a.end());` Iterators from yesterday: `a.begin() + (i + 1)` is a bookmark at the first tail element, `a.end()` is one step past the last, and the half-open range covers exactly the tail. When `i == -1`, the start is `a.begin() + 0`, the whole array. One line handles both cases.

`struct Case { ... };` inside `main`. A small bundle of two vectors, defined right where it is used. `{{1, 3, 5, 4, 2}, {1, 4, 2, 3, 5}}` fills it with brace initialization: the outer braces build a `Case`, the inner ones build each vector.

`for (const auto& c : cases)`. A range-for where `auto` lets the compiler work out the type (`Case`), and `const auto&` means "look at each case without copying it, and promise not to change it". This is the standard way to loop over a container of big objects.

`std::vector<std::vector<int>> all;` A vector whose items are vectors: a list of arrangements. `all[k] == a` compares two vectors element by element, which is exactly dictionary-order equality.

`do { ... } while (...);` Runs the body first, then checks the condition. We need this in the brute force because the starting (sorted) arrangement must be recorded before the first step. `std::next_permutation` is the standard library's version of today's algorithm; it returns `false` when it wraps around, which ends the loop.

`(k + 1) % all.size()`. The remainder operator `%` makes the index wrap: after the last arrangement comes index 0, the first.

`for (int& x : a) x = ...;` A range-for with a **non-const** reference: `x` is each actual slot of `a`, so assigning to `x` fills the vector. Without the `&`, `x` would be a copy and the vector would stay all zeros.

`std::mt19937 rng(20261028);` A random number generator (the "Mersenne Twister", from `<random>`) with a fixed seed, so every run tests the same arrays and any failure can be reproduced. `rng() % 7` gives a number from 0 to 6. Small values (`% 4`) create many duplicates, the case where off-by-one mistakes in `>` versus `>=` show up.

`std::vector<int> mine = a, lib = a;` Declares two vectors in one line, both copies of `a`.

---

4. Music: The Rolling Stones, "Gimme Shelter" (1969)

Context

"Gimme Shelter" opens *Let It Bleed*, released in December 1969. It was written by Mick Jagger and Keith Richards and produced by Jimmy Miller, who produced the Stones' great run of late-1960s and early-1970s records. Many critics and fans rank it as the best thing the band ever recorded.

The story behind it

Keith Richards came up with the guitar part in London while the mood of the times was turning dark. The late 1960s were full of war on the evening news, riots and assassinations, and the song captures that feeling of a storm about to break. It is a song about fear, and about needing somewhere safe to hide.

The track needed a powerful female voice to trade lines with Jagger. The session was in Los Angeles, late at night, and producer Jack Nitzsche suggested Merry Clayton, a gospel-trained session singer. She was called out of bed, came to the studio, and recorded her part in a few takes. On her most intense passage her voice cracks under the strain. That crack was kept, and it became the most famous moment of the record.

Let It Bleed came out within days of the Altamont free concert, where a fan was killed near the stage during the Stones' set. The Maysles brothers' documentary of that tour and that day was titled *Gimme Shelter* (1970). For many people the song became the soundtrack to the end of the 1960s dream.

Why people love it

It creates dread better than almost any rock song. Nothing about it is fast or loud at first; the tension builds through texture and voices. And Merry Clayton's performance is one of the great vocal moments in rock: a guest singer, on one night, almost stealing the song from the band.

What to listen for

- **The opening.** A shimmering, echoing guitar figure, quiet backing voices, and a scrape of percussion. It sounds like weather moving in.
- **Nicky Hopkins' piano.** The great session pianist sits underneath everything, filling the gaps.
- **The duet.** Listen to how Jagger and Clayton pass the lines back and forth, and then how Clayton takes the solo spotlight. Wait for the moment her voice breaks.
- **The harmonica.** Jagger's harmonica solo is rough and urgent, not pretty, which fits the mood.
- **The groove.** Charlie Watts keeps a steady, almost calm beat under all the chaos. That contrast is what makes the song feel so tense.